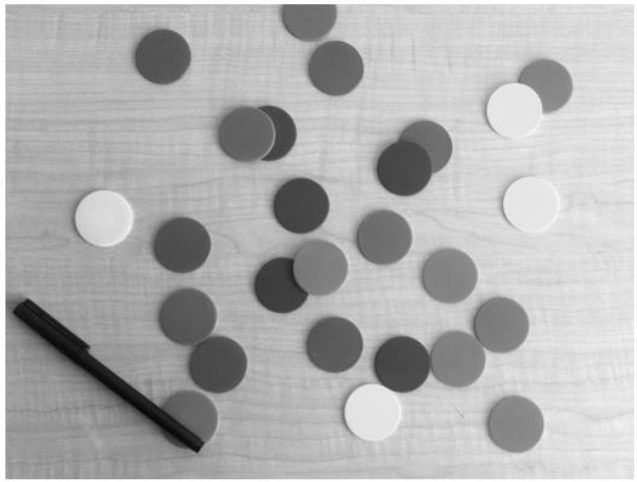
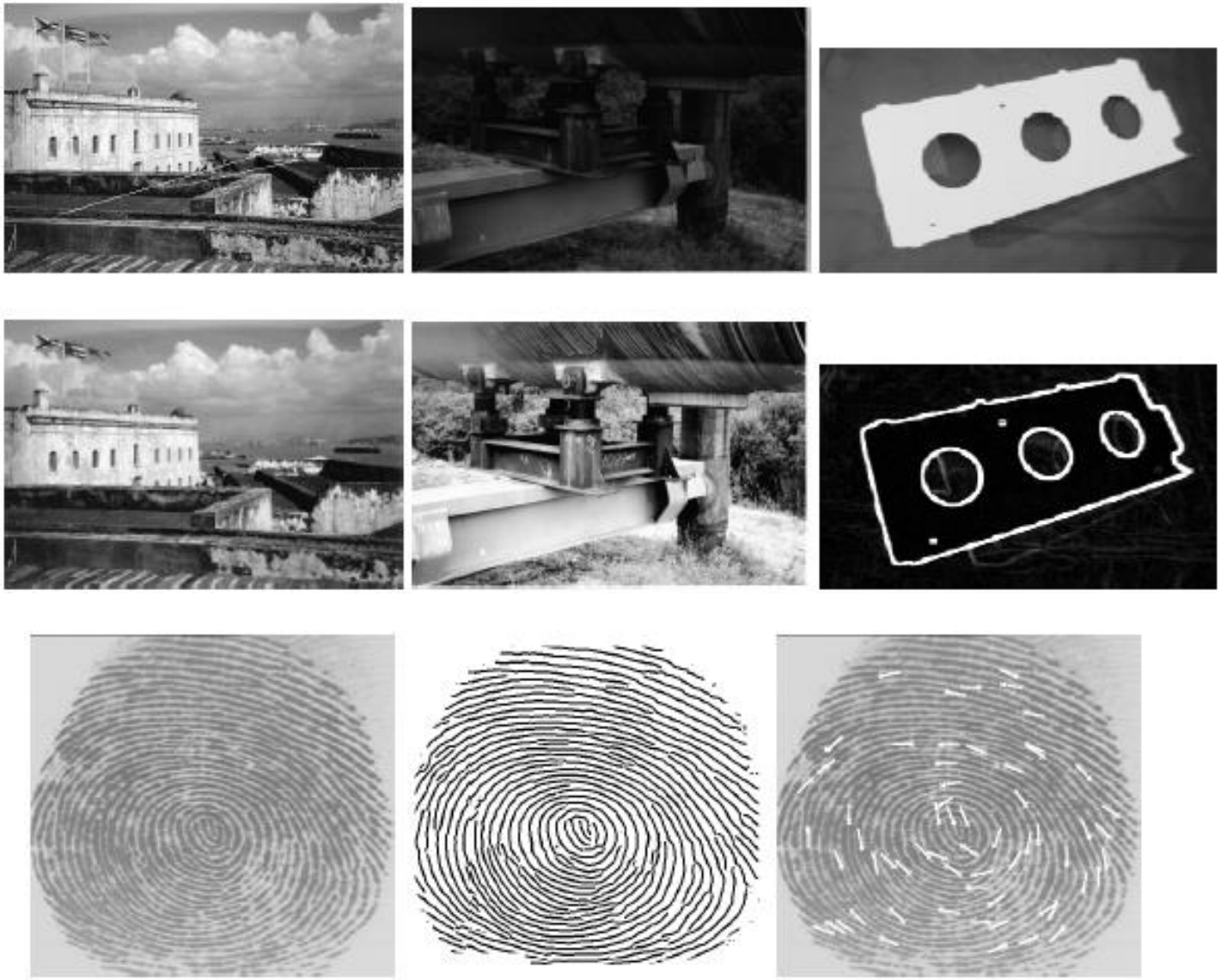
Chapter 4 – Imaging Pre-Processing

Visual Computing



The need of pre-processing



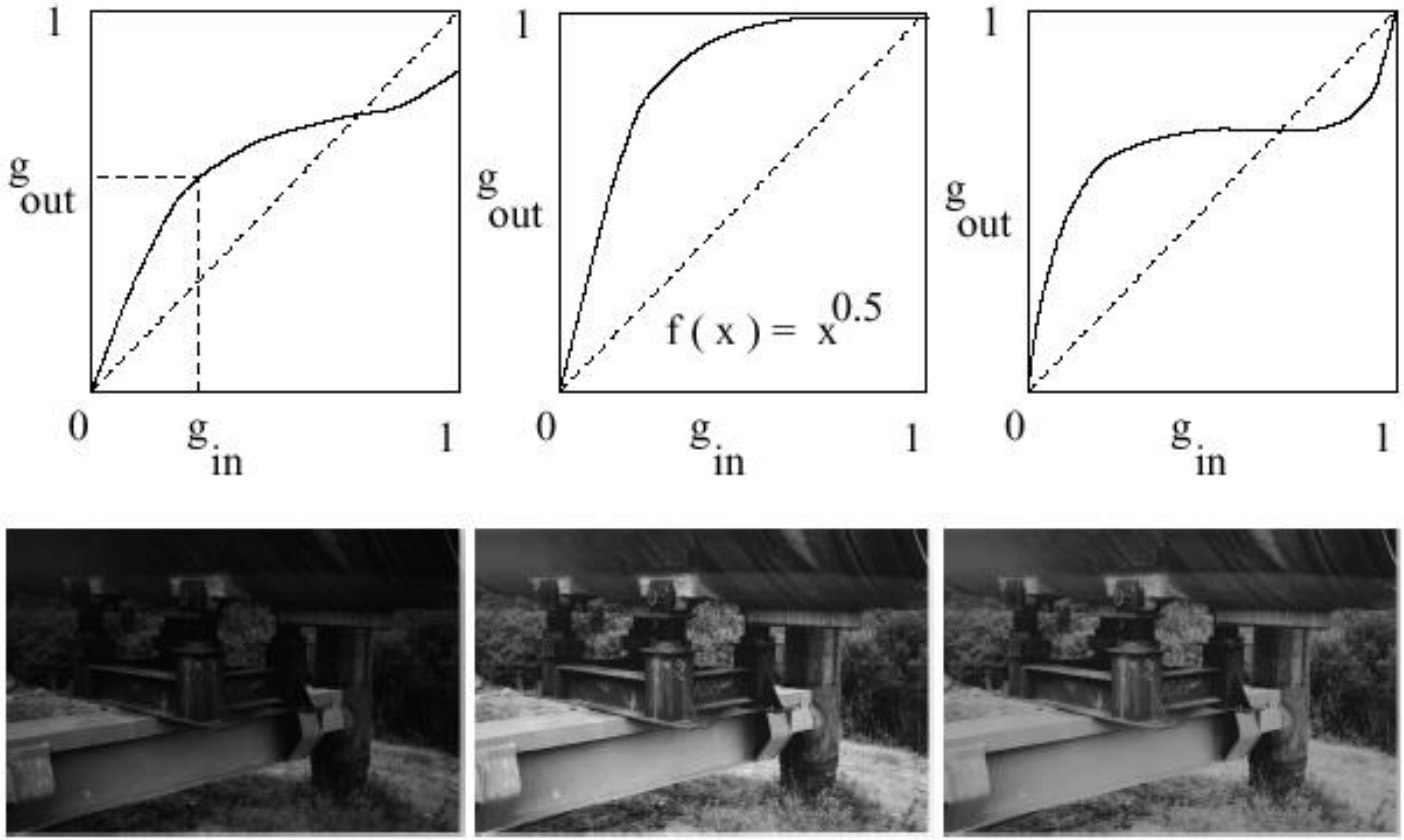
The need of pre-processing

In this chapter we are going to study

- ***Image enhacement*** (improvement of features in the image)

- ***Image restauration*** (recovery of the original contents of the image)

Gray level transformation (*Enhancement*)



- Gamma correction $\longrightarrow$ $$f(x) = O_{\min} + \frac{O_{\max} - O_{\min}}{(I_{\max} - I_{\min})^{\gamma}}(x - I_{\min})^{\gamma}$$

Gamma correction $f(x) = x^{1/\gamma}$ - Example

Input image

$f(x) = x^{0.1}$

$f(x) = x^{0.3}$

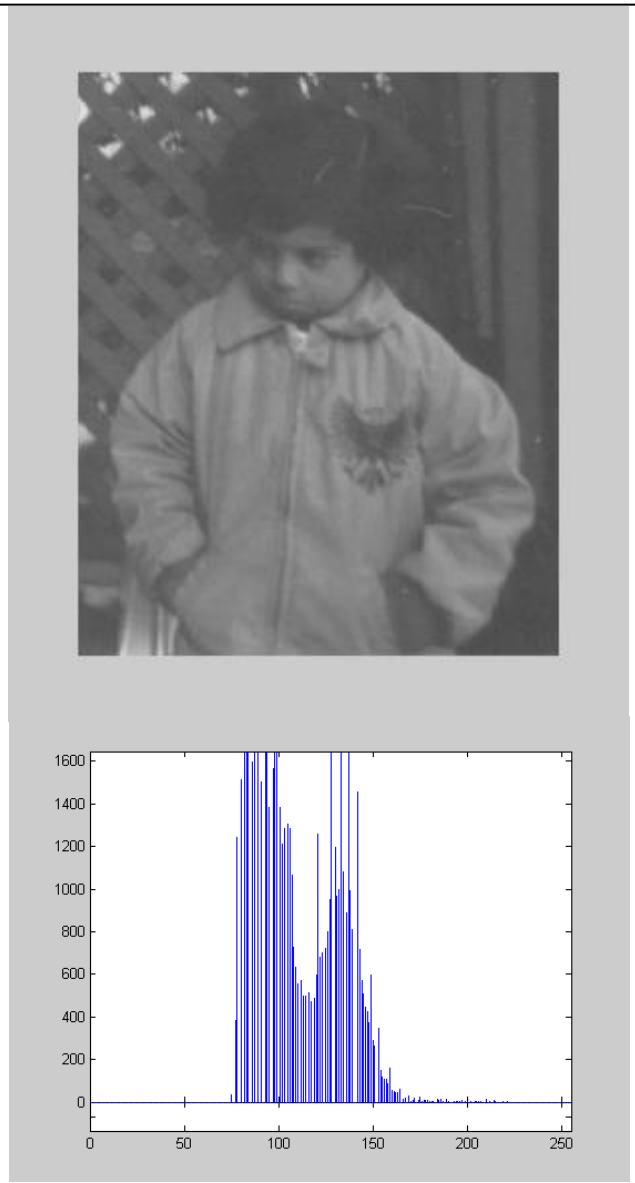
$f(x) = x^{0.5}$

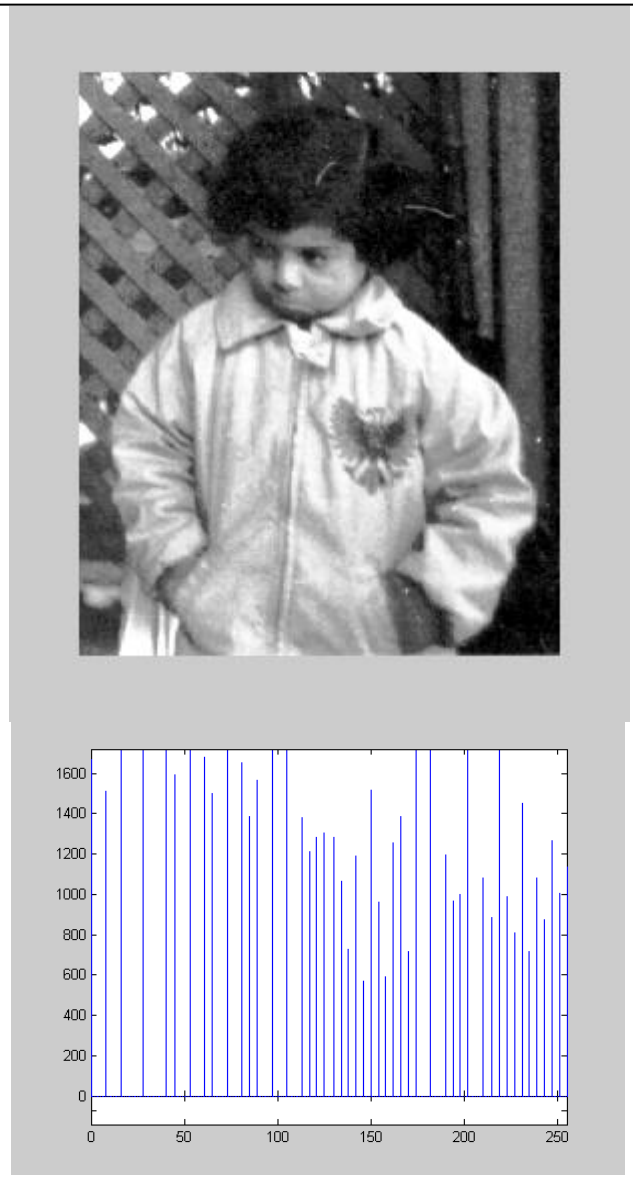
$f(x) = x^{0.7}$

Pseudo code for RGB color images

```
gammaCorrection = 1 / gamma
colour = GetPixelColour(x, y)
newRed  = 255 * (Red(colour)  / 255) ^ gammaCorrection
newGreen = 255 * (Green(colour) / 255) ^ gammaCorrection
newBlue  = 255 * (Blue(colour)  / 255) ^ gammaCorrection
PutPixelColour(x, y) = RGB(newRed, newGreen, newBlue)
```

Histogram equalization (*Enhancement*)





Histogram equalization (*Example*)

$$\begin{bmatrix} 52 & 55 & 61 & 66 & 70 & 61 & 64 & 73 \\ 63 & 59 & 55 & 90 & 109 & 85 & 69 & 72 \\ 62 & 59 & 68 & 113 & 144 & 104 & 66 & 73 \\ 63 & 58 & 71 & 122 & 154 & 106 & 70 & 69 \\ 67 & 61 & 68 & 104 & 126 & 88 & 68 & 70 \\ 79 & 65 & 60 & 70 & 77 & 68 & 58 & 75 \\ 85 & 71 & 64 & 59 & 55 & 61 & 65 & 83 \\ 87 & 79 & 69 & 68 & 65 & 76 & 78 & 94 \end{bmatrix}$$

Value	Count	Value	Count	Value	Count	Value	Count	Value	Count
52	1	64	2	72	1	85	2	113	1
55	3	65	3	73	2	87	1	122	1
58	2	66	2	75	1	88	1	126	1
59	3	67	1	76	1	90	1	144	1
60	1	68	5	77	1	94	1	154	1
61	4	69	3	78	1	104	2		
62	1	70	4	79	2	106	1		
63	2	71	2	83	1	109	1		

Given the histogram of the original image, the transformation is accomplished by using a cumulative function, that is,

$$p_x(i) = p(x = i) = n_i/n$$

PMF – probability mass function

$$cdf_x(i) = \sum_{j=0}^{i} p_x(j)$$

CDF - cumulative distribution function

Histogram equalization (*Example*)

Value	Count	Value	Count	Value	Count	Value	Count	Value	Count
52	1	64	2	72	1	85	2	113	1
55	3	65	3	73	2	87	1	122	1
58	2	66	2	75	1	88	1	126	1
59	3	67	1	76	1	90	1	144	1
60	1	68	5	77	1	94	1	154	1
61	4	69	3	78	1	104	2		
62	1	70	4	79	2	106	1		
63	2	71	2	83	1	109	1		

v, Pixel Intensity	cdf(v)	h(v), Equalized v
52	1	0
55	4	12
58	6	20
59	9	32
60	10	36
61	14	53
62	15	57
63	17	65
64	19	73
65	22	85
66	24	93
67	25	97

$$h(v) = \text{round}\left(\frac{cdf(v) - cdf_{min}}{(M \times N) - cdf_{min}} \times (L-1)\right)$$

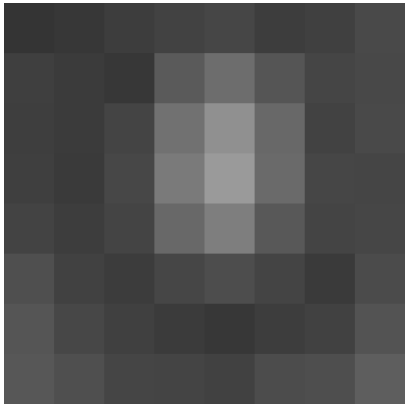
$$h(v) = \text{round}\left(\frac{cdf(v) - 1}{63} \times 255\right)$$

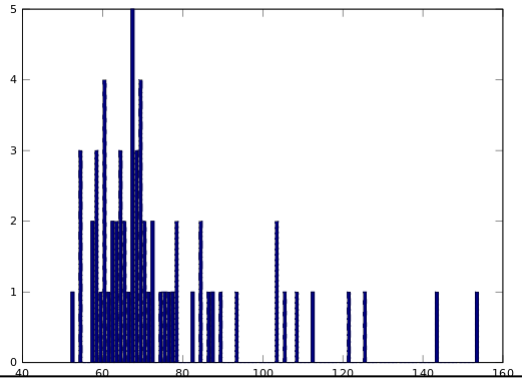
$$h(67) = \text{round}\left(\frac{25-1}{63} \times 255\right) \longrightarrow \text{round}(0.3810 \times 255) = 97.1550$$

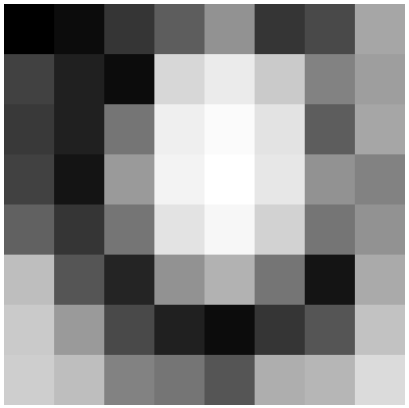
Histogram equalization (*Example*)

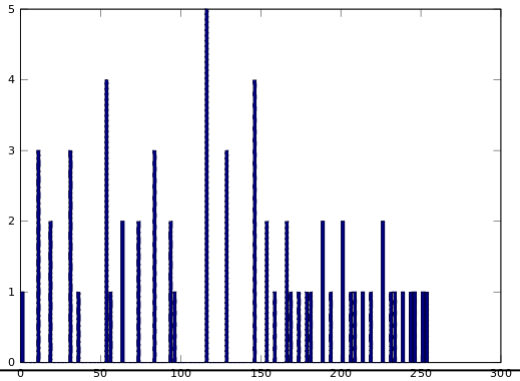
$$\begin{bmatrix} 52 & 55 & 61 & 66 & 70 & 61 & 64 & 73 \\ 63 & 59 & 55 & 90 & 109 & 85 & 69 & 72 \\ 62 & 59 & 68 & 113 & 144 & 104 & 66 & 73 \\ 63 & 58 & 71 & 122 & 154 & 106 & 70 & 69 \\ 67 & 61 & 68 & 104 & 126 & 88 & 68 & 70 \\ 79 & 65 & 60 & 70 & 77 & 68 & 58 & 75 \\ 85 & 71 & 64 & 59 & 55 & 61 & 65 & 83 \\ 87 & 79 & 69 & 68 & 65 & 76 & 78 & 94 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 12 & 53 & 93 & 146 & 53 & 73 & 166 \\ 65 & 32 & 12 & 215 & 235 & 202 & 130 & 158 \\ 57 & 32 & 117 & 239 & 251 & 227 & 93 & 166 \\ 65 & 20 & 154 & 243 & 255 & 231 & 146 & 130 \\ 97 & 53 & 117 & 227 & 247 & 210 & 117 & 146 \\ 190 & 85 & 36 & 146 & 178 & 117 & 20 & 170 \\ 202 & 154 & 73 & 32 & 12 & 53 & 85 & 194 \\ 206 & 190 & 130 & 117 & 85 & 174 & 182 & 219 \end{bmatrix}$$









Small regions removal (*Restauration*)

← **Salt & pepper** noise removal

1	1	1
1	0	1
1	1	1

⇒

1	1	1
1	1	1
1	1	1

;

0	0	0
0	1	0
0	0	0

⇒

0	0	0
0	0	0
0	0	0

X	X	X
X	L	X
X	X	X

⇒

X	X	X
X	X	X
X	X	X

;

	X	
X	L	X
	X	

⇒

	X	
X	X	X
	X	

Removal of connected components with small area →

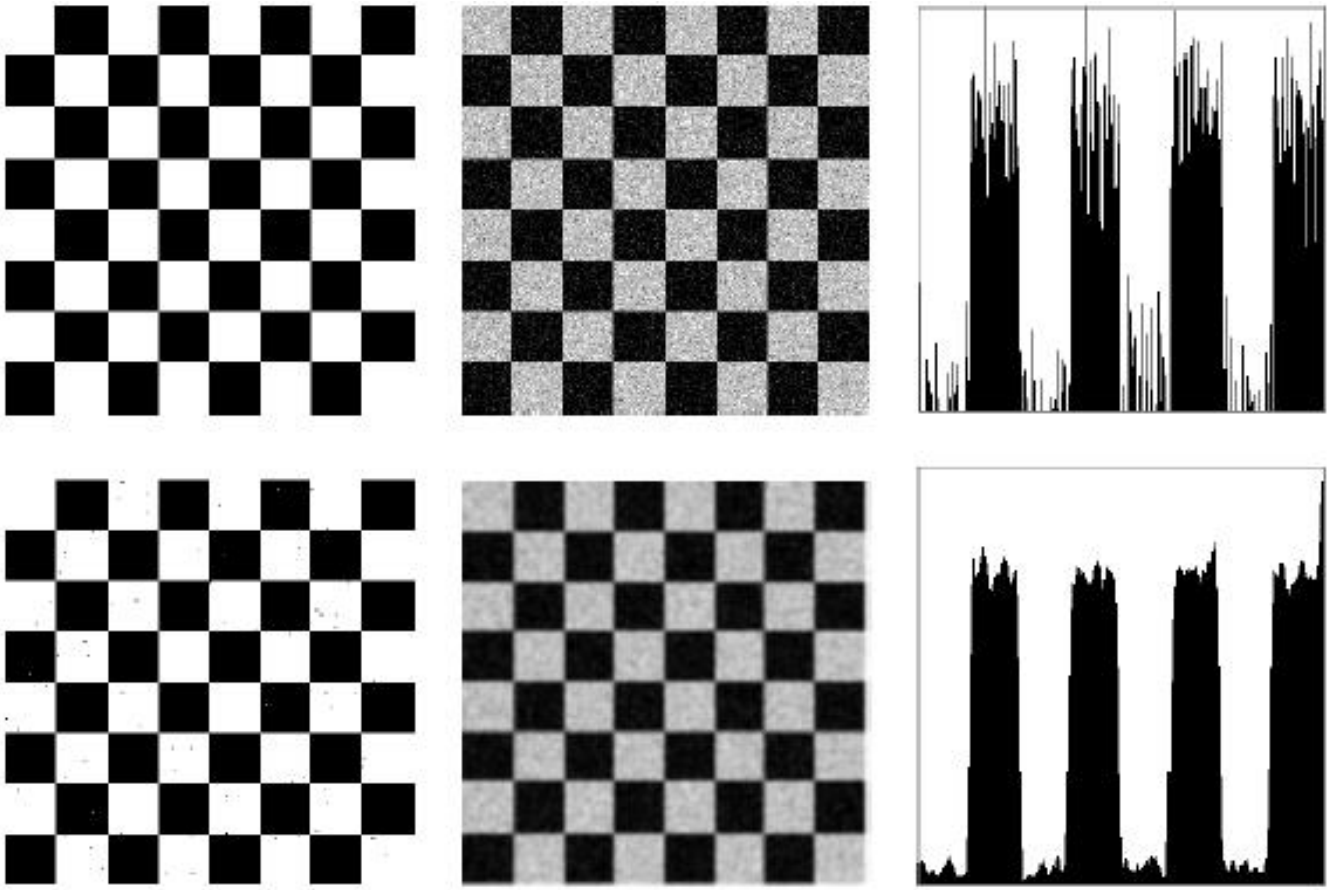
Algorithm:

If the neighborhood of an image pixel is equal to the given mask, then the pixel in the output image is replaced with the value of its neighbors.

The need of smoothness operations

image I

$I_n = I + N(0,Q)$



$I_T = I_n > T$

$I_R = I_T * \text{MeanMask}$

Algorithm: Output_Image [i,j] = *average of some neighborhood* of Input_Image [i,j]

- Smoothness (low pass filter) of an image
  - Mean filter (*box filter*)

$$O(r,c) = \left(\sum_{i=-N}^{N} \sum_{j=-N}^{N} I(r+i, c+j) \right) \Big/ (2N+1)^2$$

  - Gaussian filter

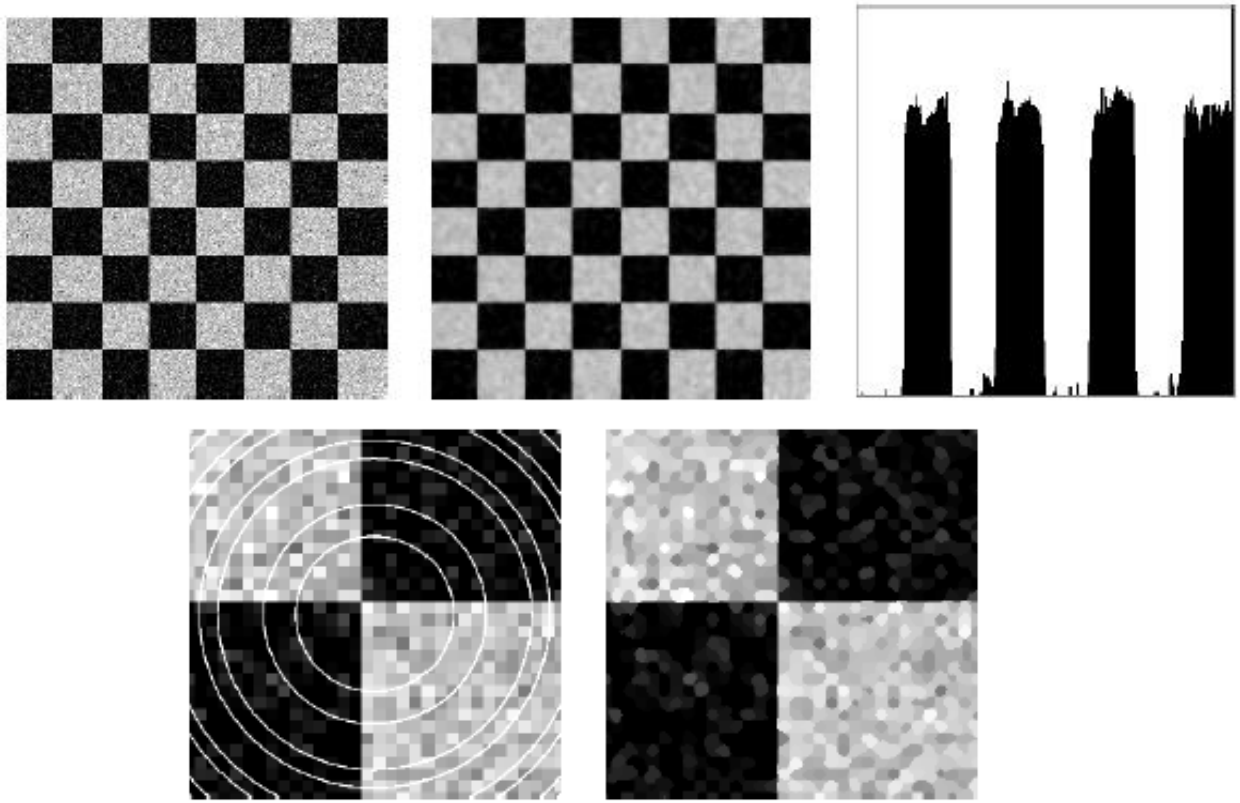
$$O(r,c) = \sum_{i=-N}^{N} \sum_{j=-N}^{N} g(i,j) I(r+i, c+j)$$

$$g(x,y) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{d^2}{2\sigma^2}}$$

$$d = \sqrt{(x - x_c)^2 + (y - y_c)^2}$$

Median filtering

- Let $\mathbf{A}[i]_{i=0,\cdots,n-1}$ be an ordered list of real numbers. The ***median*** of the set **A** is the value **A**[(n-1)/2]
  - Examples



Temporal Filtering with Median

Median filtering of the sequence

General Algorithm

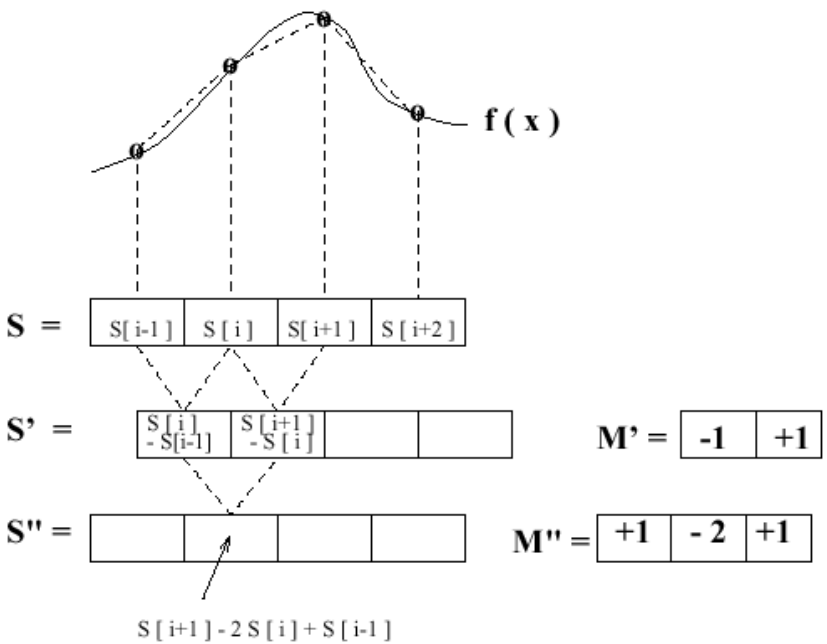
Compute output image pixel G[r,c] from neighbors of input image pixel F[r,c].

F[**r**, **c**] is an input image of MaxRow rows and MaxCol columns;
F is unchanged by the algorithm.
G[**r**, **c**] is the output image of MaxRow rows and MaxCol columns.
The border of **G** are all those pixels whose neighborhoods
are not wholly contained in **G**.
w and h are the width and height, in pixels, defining a neighborhood.

```
procedure enhance_image(F,G,w,h);
{
for r := 0 to MaxRow - 1
   for c := 0 to MaxCol - 1
      {
      if [r,c] is a border pixel then G[r,c] := F[r,c];
      else G[r,c] := compute_using_neighbors ( F, r, c, w, h );
      } ;
}
procedure compute_using_neighbors ( IN, r, c, w, h )
{
using all pixels within w/2 and h/2 of pixel IN[r,c],
compute a value to return to represent IN[r,c]
}
```

- Differential operators in 1D signals

$$f'(x_i) \approx \frac{f(x_i) - f(x_{i-1})}{x_i - x_{i-1}}$$



Input 1D mask (centered)

↓

mask M = [−1, 0, 1]

S_1			12	12	12	12	12	24	24	24	24	24
S_1	⊗	M	0	0	0	0	12	12	0	0	0	0

(a) S_1 is an upward step edge

S_2			24	24	24	24	24	12	12	12	12	12
S_2	⊗	M	0	0	0	0	-12	-12	0	0	0	0

(b) S_2 is a downward step edge

S_3			12	12	12	12	15	18	21	24	24	24
S_3	⊗	M	0	0	0	3	6	6	6	3	0	0

(c) S_3 is an upward ramp

S_4			12	12	12	12	24	12	12	12	12	12
S_4	⊗	M	0	0	0	12	0	-12	0	0	0	0

(d) S_4 is a bright impulse or "line"

One Dimensional Masks

mask M = [−1, 2, −1]

S_1			12	12	12	12	12	24	24	24	24	24
S_1	$\otimes$	M	0	0	0	0	-12	12	0	0	0	0

(a) S_1 is an upward step edge

S_2			24	24	24	24	24	12	12	12	12	12
S_2	$\otimes$	M	0	0	0	0	12	-12	0	0	0	0

(b) S_2 is a downward step edge

S_3			12	12	12	12	15	18	21	24	24	24
S_3	$\otimes$	M	0	0	0	-3	0	0	0	3	0	0

(c) S_3 is an upward ramp

S_4			12	12	12	12	24	12	12	12	12	12
S_4	$\otimes$	M	0	0	0	-12	24	-12	0	0	0	0

(d) S_4 is a bright impulse or "line"

box smoothing mask M = [1/3, 1/3, 1/3]

S_1			12	12	12	12	12	24	24	24	24	24
S_1	$\otimes$	M	12	12	12	12	16	20	24	24	24	24

(a) S_1 is an upward step edge

S_4			12	12	12	12	24	12	12	12	12	12
S_4	$\otimes$	M	12	12	12	16	16	16	12	12	12	12

(d) S_4 is a bright impulse or "line"

Gaussian smoothing mask M = [1/4, 1/2, 1/4]

S_1			12	12	12	12	12	24	24	24	24	24
S_1	$\otimes$	M	12	12	12	12	15	21	24	24	24	24

(a) S_1 is an upward step edge

S_4			12	12	12	12	24	12	12	12	12	12
S_4	$\otimes$	M	12	12	12	15	18	15	12	12	12	12

Properties

- Differential operators
  - The mask coordinates must have opposite signs to obtain a maximum response (output) whenever exists intensity transitions (contrast)
  - The sum of the values must be zero to obtain a zero response when the input region is constant.
  - The first order derivative masks produce high absolute values in points with large contrast
  - The second order derivative masks produce zero-crossing in points with large contrast
- Smoothness operators
  - The mask elements are positive and sum one, such that the output is equal to the input regions with constant intensity
  - The smoothness and noise removal are proportional to the mask dimensions
  - Abrupt transitions (step edges) are more blurred as the mask dimension increases.

- Gradient of a function

$$\nabla f = \left(\tfrac{\partial f}{\partial x}, \tfrac{\partial f}{\partial y}\right)$$

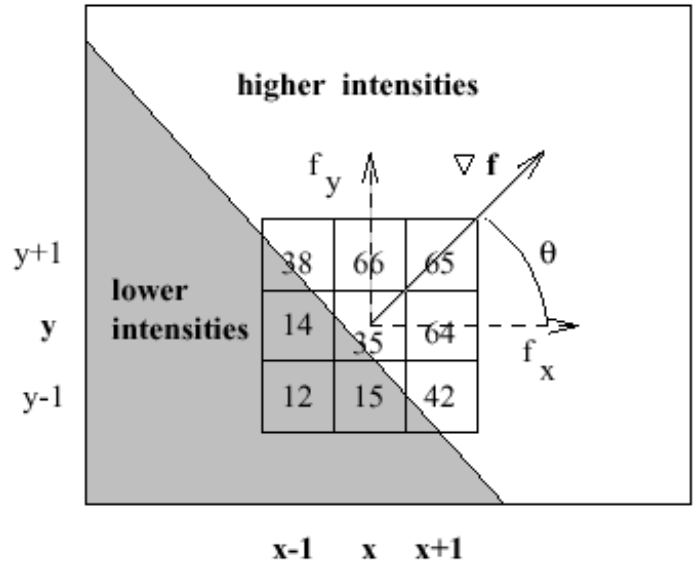
$$|\nabla f| \approx \sqrt{f_x^2 + f_y^2}$$

$$\theta \approx \tan^{-1}(f_y / f_x)$$

$$\frac{\partial f}{\partial x} \equiv f_x \approx \frac{1}{3}[(I(x+1,y) - I(x-1,y))/2 + (I(x+1,y-1) - I(x-1,y-1))/2 + (I(x+1,y+1) - I(x-1,y+1))/2]$$

$$\frac{\partial f}{\partial y} \equiv f_y \approx \frac{1}{3}[(I(x,y+1) - I(x,y-1))/2 + (I(x-1,y+1) - I(x-1,y-1))/2 + (I(x+1,y+1) - I(x+1,y-1))/2]$$

Example:



f_y = ((38-12)/2 + (66-15)/2 + (65-42)/2) / 3
= (13 + 25 + 11) / 3 = 16

f_x = ((65-38)/2 + (64-14)/2 + (42-12)/2) / 3
= (13 + 25 + 15) / 3 = 18

$\theta = \tan^{-1}(16 / 18) = 0.727$ rad
= 42 degrees

$|\nabla f| = (16^2 + 18^2)^{1/2} = 24$

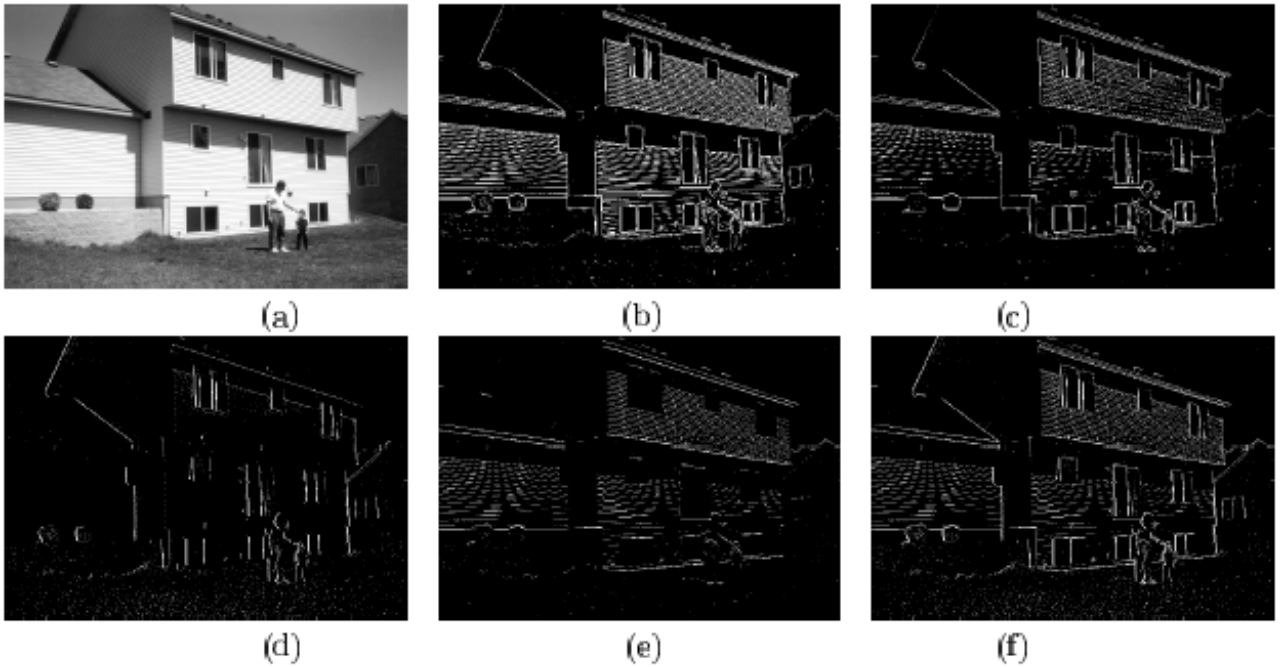
Corner Points Detectors

Prewitt: $M_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$; $M_y = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$

Sobel: $M_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$; $M_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$

Roberts: $M_x = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$; $M_y = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$

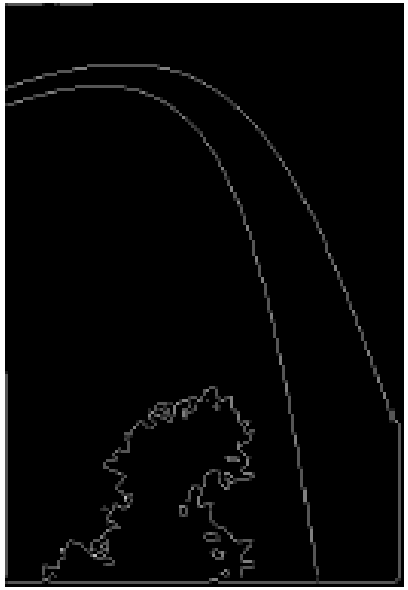
(a) Original Image
(b) Sum of the absolute values of the two Roberts mask responses
(c) Mean squared value of the two Roberts masks
(d) Absolute value of the response of y- direction edge mask [-1; 1]
(e) Absolute value of the response of x- direction edge mask [-1; 1]
(f) Images from (d) and (e)



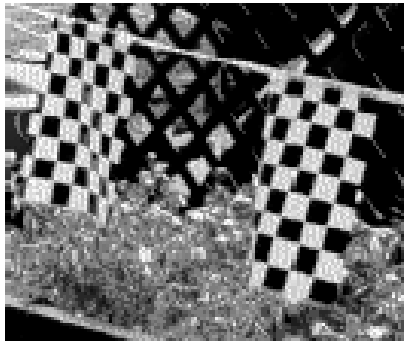
Detecting Edges with Canny

Original

Canny ($\sigma = 1$)



Canny ($\sigma = 4$)

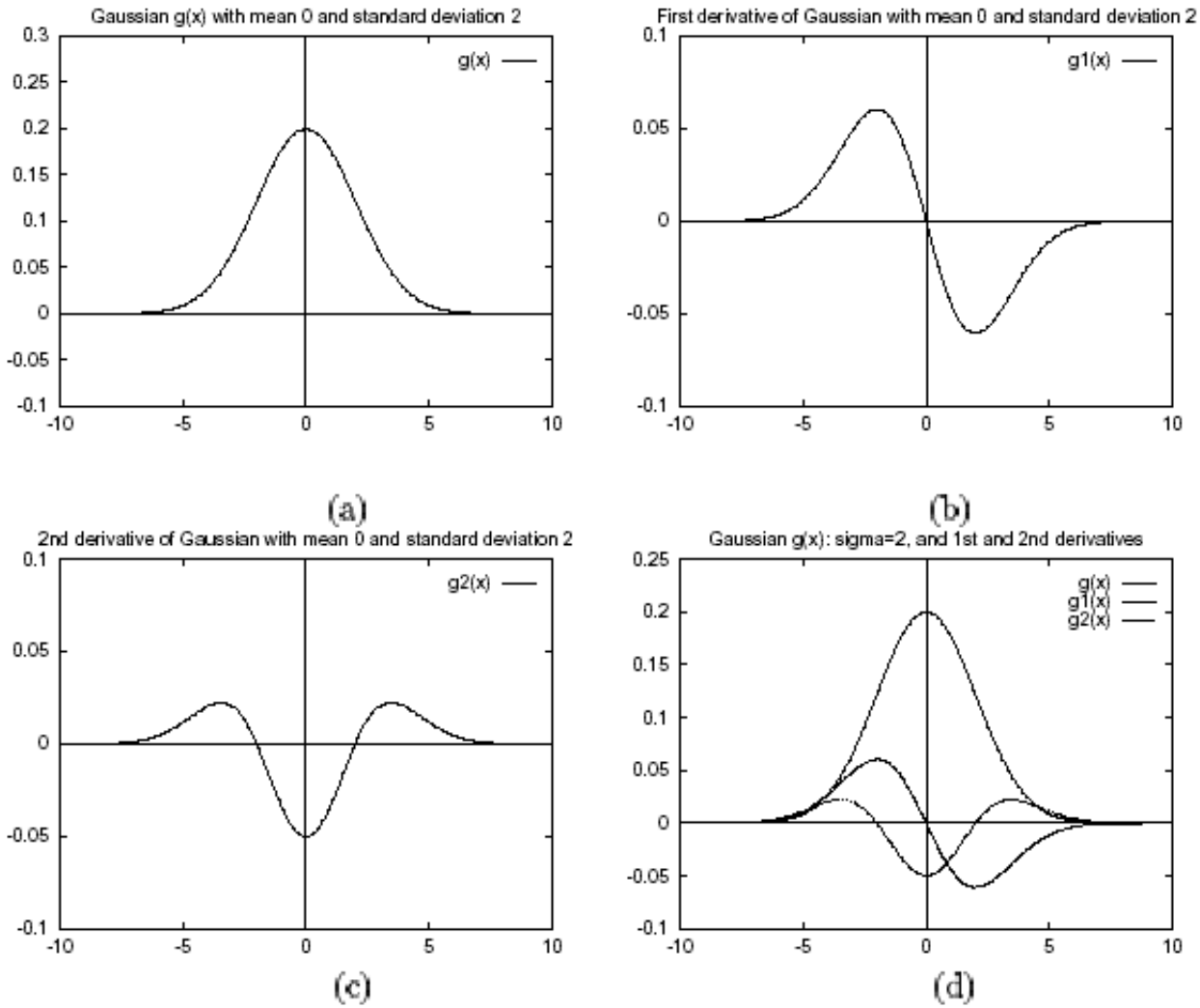


Original

Canny ($\sigma = 1$)

Roberts (20%)

Gaussian filters



1D case:

$$g(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2}{2\sigma^2}}$$

$$g'(x) = \frac{-x}{\sigma^2} g(x)$$

$$g''(x) = \left(\frac{x^2}{\sigma^4} - \frac{1}{\sigma^2} \right) g(x)$$

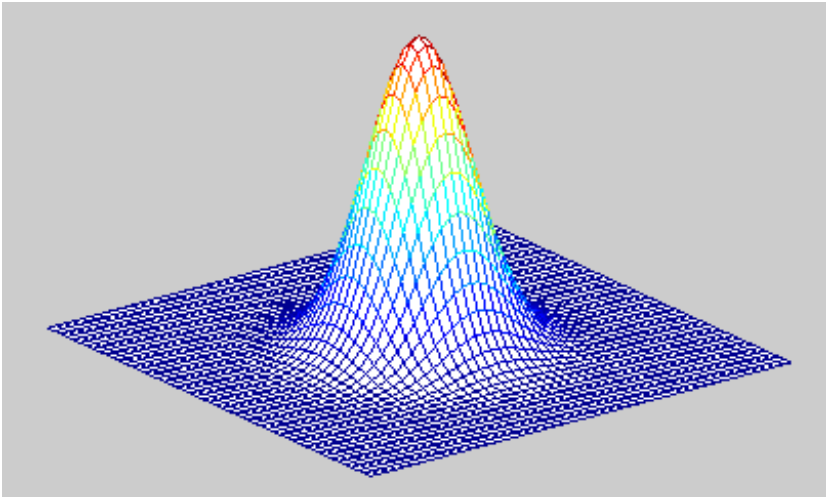
2D case:

$$h(x, y) = g(r)$$

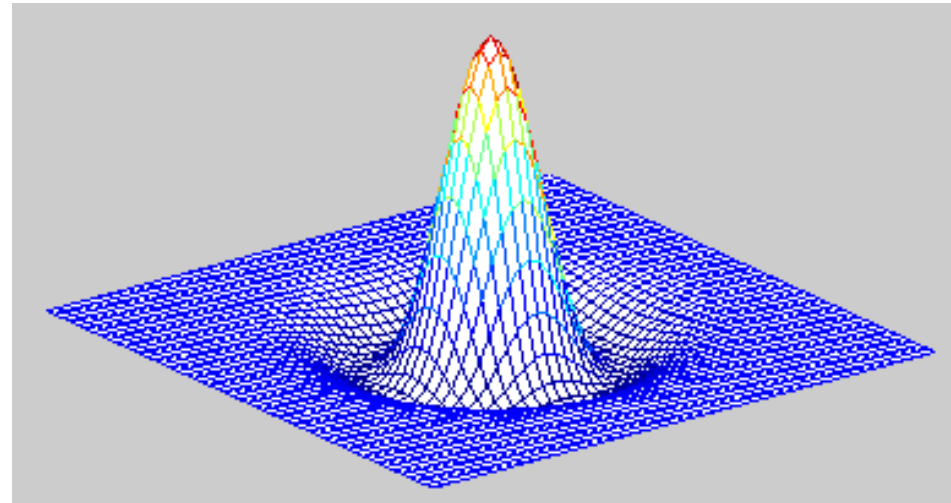
$$r = \sqrt{x^2 + y^2}$$

Edge Detection with Laplacian –LOG Filter

$g(x,y)$



$$L(x,y) = \frac{\partial^2 g(x,y)}{\partial x^2} + \frac{\partial^2 g(x,y)}{\partial y^2}$$



$-L(x,y) \longrightarrow$ Mexican hat (*sombrero*)

0	0	0	-1	-1	-2	-1	-1	0	0	0
0	0	-2	-4	-8	-9	-8	-4	-2	0	0
0	-2	-7	-15	-22	-23	-22	15	-7	-2	0
-1	-4	-15	-24	-14	-1	-14	-24	-15	-4	-1
-1	-8	-22	-14	52	103	52	-14	-22	-8	-1
-2	-9	-23	-1	103	178	103	-1	-23	-9	-2
-1	-8	-22	-14	52	103	52	-14	-22	-8	-1
-1	-4	-15	-24	-14	-1	-14	-24	-15	-4	-1
0	-2	-7	-15	-22	-23	-22	15	-7	-2	0
0	0	-2	-4	-8	-9	-8	-4	-2	0	0
0	0	0	-1	-1	-2	-1	-1	0	0	0

$\longleftarrow$ Mask 11x11 (σ^2=2)

Marr-Hildreth Theory

- The LOG filter helps to understand the Human Vision System (low level)
  - The main goal is to build the main sketch (*primal sketch: lines, edges, blobs*)
- Multi-resolution analysis
  - LOG filtering, with large σ, allows the detection of the main structures in the image, whilst the details are obtained by filtering with small σ.

original

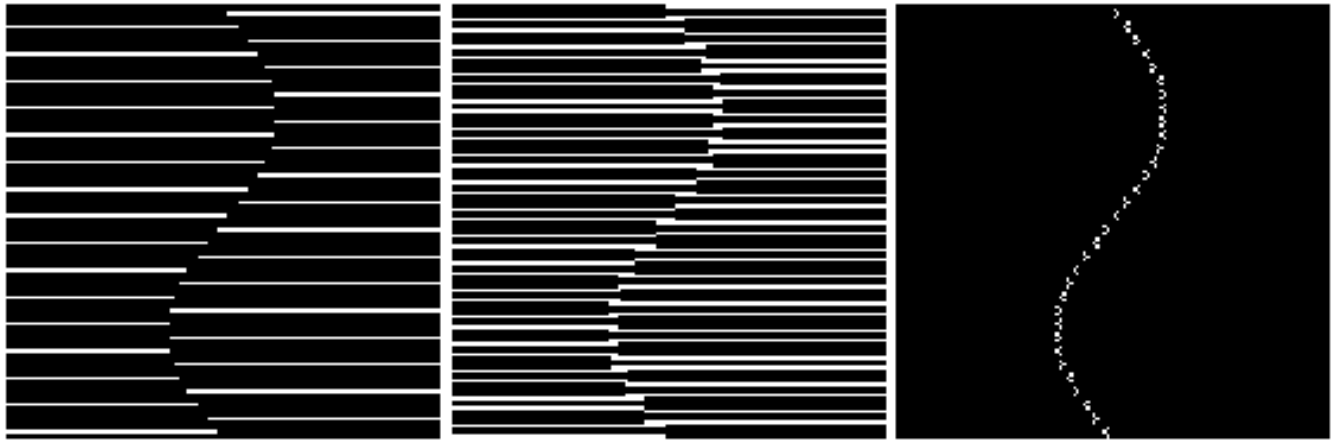
smoothed $\sigma = 4$

smoothed $\sigma = 1$

Perceptual grouping - virtual lines

Virtual line

Output of the two LOG filters



Virtual lines in real images